

Biologically Consistent Universal Differential Equations for Tumor–Immune Dynamics: Towards a more interpretable approach

Anish Sarkar¹

¹Netaji Subhash Engineering College (MAKAUT), ORCID: 0009-0009-7965-7788

ABSTRACT

We present a study of Universal Differential Equations (UDEs) for modeling tumor–immune dynamics—a progressive modeling workflow in Julia’s SciML ecosystem. We develop and compare two UDE architectures: (A) a dual-network hybrid approach that augments mechanistic Gompertz growth with neural networks to capture unmodeled immune dynamics ($R^2 = 0.897$), and (B) a structured-parameter UDE that uses neural networks to learn context-dependent parameters within classical Gompertz and Michaelis–Menten frameworks ($R^2 = 0.991$). Both variants enforce biological constraints through physics-informed loss functions. Cross-validation with validation-based early stopping yields mean $R^2 = 0.945$ on held-out groups. Our implementation leverages DifferentialEquations.jl, Flux.jl, and SciMLSensitivity.jl. Code and data are publicly available at <https://github.com/ans036/tumor-immune-ude>.

Keywords: Universal Differential Equations; Scientific Machine Learning; Tumor–Immune Dynamics; Julia; SciML; Gompertz Growth; Michaelis–Menten Kinetics; Cross-Validation.

1. Introduction

Modeling tumor–immune interactions is a long-standing challenge in computational biology [12, 7, 22]. Traditional mechanistic models offer interpretability but often underfit complex dynamics, while pure machine learning approaches lack biological constraints. Universal Differential Equations (UDEs) combine mechanistic structure with neural network flexibility, embedding trainable components within differential equation frameworks to learn missing dynamics while preserving known physics [17, 11].

Julia’s SciML ecosystem provides strong infrastructure for UDE development through DifferentialEquations.jl [16], Flux.jl [9], and SciMLSensitivity.jl for adjoint-based gradient computation [4, 10, 15]. While UDEs have been applied in hydrology and epidemiology [20, 1, 6], their application to tumor–immune modeling with rigorous cross-validation remains underexplored.

This paper presents a progressive modeling workflow for tumor–immune dynamics in Julia’s SciML ecosystem, with the following contributions: (i) a dual-network hybrid UDE that augments Gompertz growth with learned immune dynamics, (ii) a structured-parameter UDE where neural networks learn context-dependent parameters within classical frameworks, (iii) cross-validation with validation-based early stopping for robust evaluation, and (iv) a fully reproducible Julia implementation.

2. Statement of need

Researchers modeling tumor–immune interactions need tools that balance mechanistic interpretability with predictive accuracy. Existing approaches are either purely mechanistic models that underfit or black-box methods that lack biological grounding. Our Julia package provides a UDE-based framework that integrates domain knowledge through physics-informed constraints, learns interpretable biological parameters, and provides cross-validated forecasting—all within Julia’s SciML ecosystem.

3. State of the field

Classical tumor–immune models such as the Kuznetsov–Taylor model [12] and the de Pillis–Radunskaya system [7] provide interpretable frameworks but are parameter-heavy and difficult to calibrate with sparse data. The Gompertz model [22] describes tumor growth kinetics well but lacks immune interaction terms. Neural ODEs [5] and Physics-Informed Neural Networks [18] have shown promise in scientific applications. UDEs [17] enable hybrid mechanistic-neural models, though existing applications in biology [20, 1, 6] have not specifically targeted tumor–immune dynamics with cross-validated evaluation. Our work applies UDEs to this domain with biologically-constrained Gompertz growth and Michaelis–Menten immune killing, using a progressive workflow from hybrid to structured-parameter models.

4. Methodology

4.1 Universal Differential Equations

A UDE decomposes the dynamics of a system $\frac{d\mathbf{u}}{dt} = f(\mathbf{u}, t, \boldsymbol{\theta})$ as:

$$f(\mathbf{u}, t, \boldsymbol{\theta}) = f_{\text{known}}(\mathbf{u}, t, \boldsymbol{\theta}_{\text{mech}}) + f_{\text{unknown}}(\mathbf{u}, t, \boldsymbol{\theta}_{\text{nn}}) \quad (1)$$

where f_{known} encodes mechanistic knowledge and f_{unknown} captures unmodeled dynamics through neural networks [13, 14, 8]. Gradients flow through both components via differentiable programming, enabling end-to-end optimization.

4.2 Biological Foundation

Tumor growth follows the Gompertz model:

$$\frac{dV}{dt} = rV \ln\left(\frac{K}{V}\right) \quad (2)$$

where $V(t)$ is tumor volume, r is the growth rate, and K is the carrying capacity. Immune-mediated tumor suppression follows

Michaelis–Menten kinetics:

$$R_{\text{kill}}(V, I) = \frac{c \cdot V \cdot I(t)}{h + V} \quad (3)$$

where R_{kill} denotes the immune killing rate, $I(t)$ is immune cell concentration, c is the maximum killing rate constant (units: day^{-1}), and h is the half-saturation constant (volume at which killing reaches half its maximum rate).

4.3 Phase 1: Dual-Network Hybrid UDE

The Gompertz model (Eq. 2) describes tumor growth in isolation but does not account for immune cell interactions. Classical models like Kuznetsov–Taylor [12] address this through explicit effector-cell compartments, but require many parameters and strong mechanistic assumptions. Our hybrid approach instead uses neural networks to *learn* the immune interaction and residual temporal dynamics directly from data, while retaining the well-established Gompertz backbone. This avoids prescribing the functional form of immune suppression a priori, allowing the data to reveal which interactions matter. The hybrid UDE augments Gompertz growth with two neural networks:

$$\frac{dV}{dt} = \underbrace{r_g V \ln \left(\frac{K_g}{V} \right)}_{\text{Gompertz}} - \underbrace{\alpha_g \phi_{\text{IRN}}(V_n, I_n)}_{\text{Learned Immune}} + \underbrace{\beta w(V_n) \phi_{\text{TACN}}(V_n, t_n)}_{\text{Temporal Correction}} \quad (4)$$

where $\phi_{\text{IRN}} : \mathbb{R}^2 \rightarrow [0, 1]$ is an Immune Response Network (sigmoid output), $\phi_{\text{TACN}} : \mathbb{R}^2 \rightarrow [-1, 1]$ is a Time-Aware Correction Network (tanh output), $w(V_n)$ is a sigmoid gate suppressing corrections at small volumes, and $r_g, K_g, \alpha_g, \beta$ are learnable parameters.

4.4 Phase 2: Structured-Parameter UDE

Phases 1 and 2 form a *sequential* workflow: the hybrid model (Phase 1) serves as an exploratory tool whose learned network behaviors reveal that immune killing and temporal corrections are the dominant missing dynamics. Phase 2 then incorporates these insights into a more constrained architecture. Specifically, the structured-parameter UDE preserves the classical Gompertz + Michaelis–Menten equation structure but uses neural networks to make all parameters context-dependent:

$$\frac{dV}{dt} = \underbrace{r(V, t, I) \cdot V \ln \left(\frac{K(V, t, I)}{V} \right)}_{\text{Dynamic Gompertz}} - \underbrace{\frac{c(V, t, I) \cdot V \cdot I(t)}{h(V, t, I) + V}}_{\text{Dynamic Michaelis–Menten}} \quad (5)$$

where neural networks learn the mappings:

$$(r, K) = \text{NN}_{\text{growth}}(V_n, t_n, I_n), \quad (c, h) = \text{NN}_{\text{immune}}(V_n, t_n, I_n) \quad (6)$$

with activation functions enforcing bounds: $r \in [r_{\min}, r_{\max}]$, $K \geq 1.1V$, $c \in [c_{\min}, c_{\max}]$, $h \in [h_{\min}, h_{\max}]$.

Extrapolation note. Because the neural networks in Eq. 5 take normalized time t_n as a direct input, learned parameter functions may not generalize when t extends far beyond the training range. The sigmoid-bounded activations mitigate divergence by constraining outputs to biologically plausible intervals, but long-term forecasts should be interpreted as extrapolations with increasing uncertainty (see Section 8.1).

Terminology note. We call this the “structured-parameter” UDE because, although all four biological parameters are learned by neural networks (making this a neural model), the outputs are channeled

through the classical Gompertz and Michaelis–Menten equation structure. The neural networks do not directly specify the dynamics; rather, they parameterize a fixed mechanistic template. This preserves the interpretability of each parameter’s biological role while allowing context-dependent variation.

4.5 Physics-Informed Constraints

We enforce biological priors through several loss terms. A consistency prior ensures immune activity does not increase tumor volume:

$$\mathcal{L}_{\text{consistency}} = \lambda_{\text{cons}} \sum_{g,j} \max \left(0, \hat{V}_g^{\text{immune}}(t_{g,j}) - \hat{V}_g^{\text{no-immune}}(t_{g,j}) + \varepsilon \right)^2 \quad (7)$$

where $\hat{V}_g^{\text{immune}}(t_{g,j})$ denotes the model-predicted tumor volume (hat indicates a predicted quantity) for group g at time point $t_{g,j}$ with immune effects included, and $\hat{V}_g^{\text{no-immune}}(t_{g,j})$ is the corresponding counterfactual prediction with immune cell input set to zero. The subscript g indexes over tumor groups and j indexes over observation time points within each group. The margin $\varepsilon = 0.05$ ensures a minimum separation. This term penalizes any instance where the immune-present trajectory exceeds the immune-absent trajectory, enforcing the biological prior that immune activity should suppress—not enhance—tumor growth.

A physics prior aligns learned immune parameters with observed killing rates via rank correlation:

$$\mathcal{L}_{\text{physics}} = \lambda_{\text{phys}} (1 - \rho_{\text{rank}}(I_{\text{static}}, \hat{c}_{\text{kill}}))^2 \quad (8)$$

The Phase 1 (hybrid UDE) total loss combines volume fitting, derivative matching, a smoothness penalty, physics priors, negativity penalties, and L_2 regularization:

$$\begin{aligned} \mathcal{L}^{(1)} = & w_{\text{vol}} \lambda_{\text{vol}} \mathcal{L}_{\text{volume}} + w_{\text{deriv}} \lambda_{\text{deriv}} \mathcal{L}_{\text{deriv}} \\ & + \lambda_{\text{smooth}} \mathcal{L}_{\text{smooth}} + \lambda_{\text{phys}} \mathcal{L}_{\text{physics}} \\ & + \lambda_{\text{neg}} \mathcal{L}_{\text{negativity}} + \lambda_{\text{reg}} \mathcal{L}_{\text{reg}} \end{aligned} \quad (9)$$

where the individual terms are:

$\mathcal{L}_{\text{volume}} = \frac{1}{N} \sum_{g,j} (\hat{V}_g(t_{g,j}) - V_g^{\text{obs}}(t_{g,j}))^2$: mean squared error between predicted and observed tumor volumes (N = total data points across groups);

$\mathcal{L}_{\text{deriv}} = \frac{1}{M} \sum_{g,j} (\dot{\hat{V}}_g(t_{g,j}) - \dot{V}_g^{\text{obs}}(t_{g,j}))^2$: derivative-matching loss, where $\dot{V}^{\text{obs}}$ is estimated from observations by central finite differences and $\dot{\hat{V}}$ is the model RHS evaluated at the predicted state. This term encourages the learned dynamics, not just the integrated trajectory, to match the data;

$\mathcal{L}_{\text{smooth}} = \sum_{g,j} (\hat{V}_g(t_{j+1}) - 2\hat{V}_g(t_j) + \hat{V}_g(t_{j-1}))^2$: a discrete second-difference smoothness penalty on the predicted trajectory, which discourages spurious high-frequency oscillations introduced by the neural sub-networks (this corresponds to the constant λ_{SMOOTH} in the source code);

$\mathcal{L}_{\text{negativity}} = \sum_{g,j} \max(0, -\hat{V}_g(t_{g,j}))^2$: penalizes non-physical negative volume predictions;

$\mathcal{L}_{\text{reg}} = \|\theta_{\text{nn}}\|_2^2$: L_2 weight decay on neural-network parameters.

The two *phase-dependent* weights w_{vol} and w_{deriv} implement curriculum-style scheduling: during the AdamW phase (phase = 1) we use $w_{\text{vol}} = 1.0$, $w_{\text{deriv}} = 1.0$, prioritizing trajectory fitting; during the L-BFGS refinement phase (phase = 2) we shift to $w_{\text{vol}} = 0.8$, $w_{\text{deriv}} = 1.5$, increasing the influence of the derivative-matching term on local refinement. The fixed weights are $\lambda_{\text{vol}} = 1.0$,

$\lambda_{\text{deriv}} = 10^{-2}$, $\lambda_{\text{smooth}} = 10^{-3}$, $\lambda_{\text{phys}} = 0.1$, $\lambda_{\text{neg}} = 10^{-3}$, $\lambda_{\text{reg}} = 10^{-5}$, selected by grid search on training-loss convergence and biological consistency. The Phase 2 (structured-parameter UDE) loss replaces the smoothness and derivative terms with the consistency prior $\mathcal{L}_{\text{consistency}}$ defined in Eq. 7, retaining $\mathcal{L}_{\text{volume}}$, $\mathcal{L}_{\text{physics}}$, $\mathcal{L}_{\text{negativity}}$, and $\mathcal{L}_{\text{reg}}$. The naming used here—*smoothness*, *consistency*, etc.—corresponds directly to the loss terms in the source code (`src/JuliaconSubmission.jl` and `src/FutureWorks.jl`); the `smooth_penalty` variable in the code is the smoothness term above, and the `consistency_loss` variable in the Phase 2 code is the consistency prior.

4.6 Training Protocol

Training uses a two-phase approach: (1) AdamW with learning rate 10^{-3} , cosine decay, and up to 5 000 iterations with validation-based early stopping, followed by (2) L-BFGS refinement from the best AdamW parameters (up to 500 iterations). Early stopping uses a patience of 200 iterations, meaning training halts if the validation loss does not improve for 200 consecutive iterations; the model checkpoint with the lowest validation loss is retained. L-BFGS terminates when the gradient norm falls below 10^{-8} (the default convergence criterion in `Optim.jl`) or the maximum iteration count is reached, whichever comes first.

Note on AdamW iteration budget. The 5 000-iteration upper bound is intentionally loose: in practice, the validation-based early-stopping rule typically terminates the AdamW phase well before the cap is reached (the loss curves in Figures 1–5 flatten by roughly ~ 500 – $1\,000$ iterations). Because the L-BFGS phase initializes from the lowest-validation-loss checkpoint, retaining the high cap incurs little additional cost and provides a safety margin for groups whose loss landscape has not yet stabilized; tightening the cap to ~ 500 – $1\,000$ iterations would yield comparable final fits at lower training cost. We prioritized robustness over efficiency given the relatively short per-fold training times (~ 15 – 30 minutes).

Hyperparameter bounds. The parameter ranges $r \in [0.01, 2.0]$ day^{-1} , $c \in [0.001, 10.0]$ day^{-1} , and $h \in [0.01, 5.0]$ (normalized volume units) were chosen based on published estimates from the tumor-modeling literature. Growth rates are bounded by experimentally observed doubling times for solid tumors [22]. Killing rate and half-saturation bounds span the physiologically plausible range reported in Kuznetsov et al. [12] and de Pillis et al. [7], scaled to our normalized volume units. Sigmoid activations map unbounded neural network outputs into these intervals.

4.7 Neural Network Architecture

Both variants use custom residual blocks with layer normalization: $\text{ResBlock}(\mathbf{x}) = \text{Activation}(\mathbf{x} + \mathbf{W}_2\sigma(\text{LayerNorm}(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)) + \mathbf{b}_2)$. Phase 1 networks take 2 inputs $\rightarrow$ 32 hidden $\rightarrow$ 2 ResBlocks $\rightarrow$ 16 hidden $\rightarrow$ 1 output. Phase 2 networks take 3 inputs $\rightarrow$ 32 hidden $\rightarrow$ 2 ResBlocks $\rightarrow$ parameter-specific outputs. SELU activation is used for hidden layers [21].

4.8 Computational Implementation

We use `Tsit5()` with `InterpolatingAdjoint()` for gradient computation, with tolerances `abstol` = 10^{-8} , `reitol` = 10^{-8} . Training employs $N = 3$ random initializations (seeds 1042, 2042, 3042) to mitigate local minima. Each initialization undergoes the full AdamW + L-BFGS training protocol, and the final model is selected as the one achieving the lowest total training loss (Eq. 9) across all restarts.

5. Experimental Setup

5.1 Data

We use the publicly available murine tumor–immune dataset released by Atsou et al. [2, 3] alongside their size- and space-structured tumor–immune interaction model. The data are distributed under a Creative Commons Zero (CC0) license at <https://github.com/kevin-atsou/tumorgrowth/tree/main/data> and originate from in vivo experiments using the mSCC38 squamous cell carcinoma cell line in FVB/N wild-type mice, described in detail in [3] (Materials, Section 2.3). We obtained both CSVs unmodified from that repository and re-host them at `Data/` in our own repository for convenience.

The temporal dataset (`tumor_time_to_event_data.csv`) contains longitudinal tumor-volume measurements grouped by (KineticID, TumorID) pairs, where the KineticID values {C2, C3, C4} index three experimental immune-cell conditions and the TumorID values {T1, ..., T5} index five tumor replicates / scenarios. For each pair the file records: observation time (days), tumor volume (mm^3), and the concurrently measured immune cell count. Measurements were taken at days $t \in \{14, 15, 21, 22, 28, 29, 36\}$, giving up to 7 time points per group (some groups have fewer due to missing or excluded measurements). A static dataset (`tumor_volume_vs_Im_cells_rate.csv`) provides 39 paired measurements of tumor volume and immune killing rate (rank correlation $r = -0.82$), used for physics-informed constraints. The reported forecast windows in Section 7 (e.g. days 15–36 in Figures 9–10) correspond to the empirical observation interval of this dataset; the lower bound is the first observation time and the upper bound is the last.

Tumor volumes are normalized by 1000 mm^3 for numerical stability. Sparse immune measurements are interpolated using cubic splines. Data is grouped by {KineticID, TumorID}, excluding groups with fewer than 2 time points.

5.2 Cross-Validation Design

Each tumor group is identified by a pair {KineticID, TumorID}: the KineticID (C2, C3, C4) denotes the immune condition (e.g., concentration level of administered immune cells), and the TumorID (T1–T5) denotes the tumor replicate or scenario. The notation “C2-T1” thus refers to the trajectory measured under immune condition C2 for tumor scenario T1.

We employ a leave-one-group-out cross-validation scheme. For each held-out group (e.g., C2-T1), the training set comprises: (a) data from all *other* immune conditions for the same tumor type (e.g., C3-T1 and C4-T1), and (b) data from the held-out condition for all *other* tumor types (e.g., C2-T2, C2-T3, etc.). This design tests the model’s ability to generalize across immune conditions for a given tumor type. Five groups are held out for evaluation: C2-T1, C2-T2, C2-T3, C2-T4, and C4-T5, selected to span different immune conditions and tumor scenarios.

6. Software design

The implementation is a modular Julia package. `EnhancedPINODE.jl` implements the dual-network hybrid UDE (Phase 1). `PureMechanisticUDE.jl` implements the structured-parameter UDE (Phase 2). Neural networks use `Flux.jl` [9], ODE integration uses `DifferentialEquations.jl` [16] with `SciMLSensitivity.jl`, and optimization uses `Optimization.jl` [15]. The `test/` directory contains unit tests with CI via GitHub Actions. The package is avail-

able at <https://github.com/ans036/tumor-immune-ude> and requires Julia 1.9+.

7. Results

7.1 Model Performance Comparison

Table 1 summarizes the progression from classical baselines to our UDE approaches. We compare against three classical mechanistic models: Gompertz, Kuznetsov–Taylor [12], and de Pillis–Radunskaya [7]. We note that this comparison is limited to classical ODE models from the tumor-modeling literature; we do not compare against modern neural baselines such as Neural ODEs [5], latent ODEs, or Gaussian process models, which represents a limitation of this study (see Section 8.1).

7.2 Phase 1: Hybrid UDE

The hybrid UDE converges stably through both AdamW and L-BFGS phases, achieving $R^2 = 0.897$ (Figure 1). The Immune Response Network learns biologically meaningful patterns: volume-dependent activation consistent with antigen presentation, saturation at high immune fractions, and threshold effects at low immune levels (Figure 2). The Time-Aware Correction Network captures temporal effects not present in the mechanistic backbone (Figure 3). While the correction magnitude does increase over time—reflecting growing discrepancy between the Gompertz backbone and observed dynamics—the *rate of increase* in correction diminishes at late times, indicating that the learned correction approaches an asymptotic regime rather than diverging. We use “stabilization” to refer to this plateauing behavior of the correction’s derivative, not a decrease in absolute magnitude. Learned parameters exhibit group-specific variation (Figure 4): growth rates span $r \in [0.10, 0.34] \text{ day}^{-1}$ across groups, with the C4/T5 group learning the highest rate ($r = 0.34 \text{ day}^{-1}$) consistent with its rapid observed growth. Immune strength parameters α range from 0.11 to 0.38, showing group-level variation that reflects the interplay between immune cell counts and tumor-specific sensitivity rather than a simple monotonic relationship.



Fig. 1: Phase 1 training loss convergence through AdamW and L-BFGS phases.

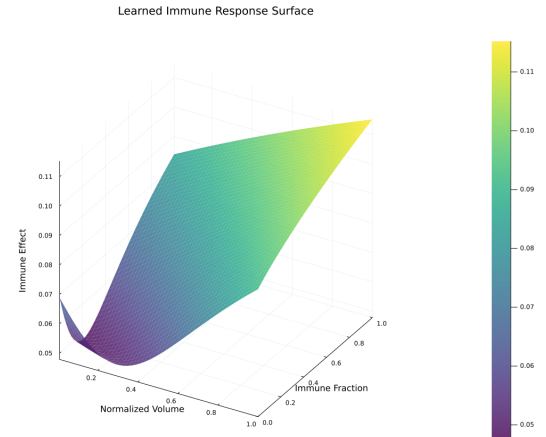


Fig. 2: Learned Immune Response Network surface showing volume-dependent activation and saturation behavior.

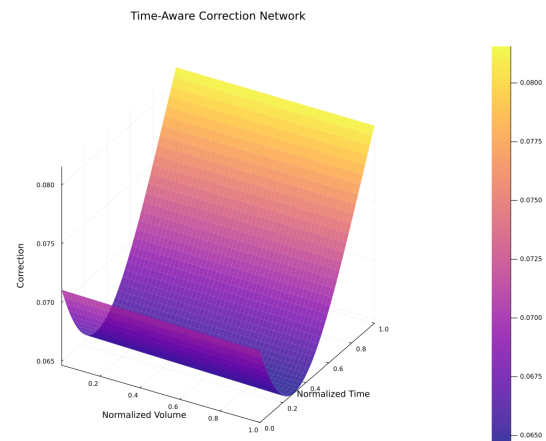


Fig. 3: Time-Aware Correction Network surface showing temporal adjustments.

Model	Architecture	Neural Integration	R^2
Gompertz Baseline	$rV \ln(K/V)$	None	0.763
Kuznetsov–Taylor	Multi-effector ODEs	None	0.821
de Pillis–Raduns kaya	Immune + therapy terms	None	0.845
Hybrid UDE (Phase 1)	Eq. 4	IRN + TACN	0.897
Structured-Param. UDE (Phase 2)	Eq. 5	Parameter learning	0.991

Table 1. : Model comparison showing progression from classical baselines to both UDE approaches (Phase 1: Hybrid UDE; Phase 2: Structured-Parameter UDE). Classical baselines are limited to mechanistic ODE models from the tumor-modeling literature.

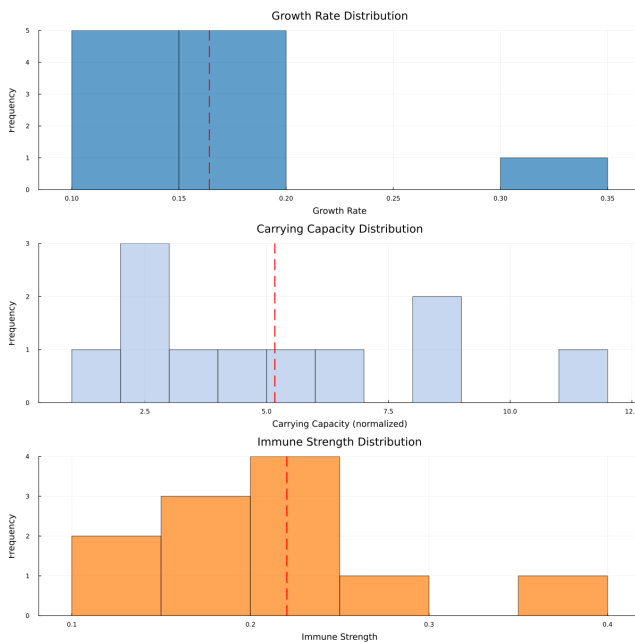


Fig. 4: Distribution of learned parameters across tumor groups in the hybrid model.

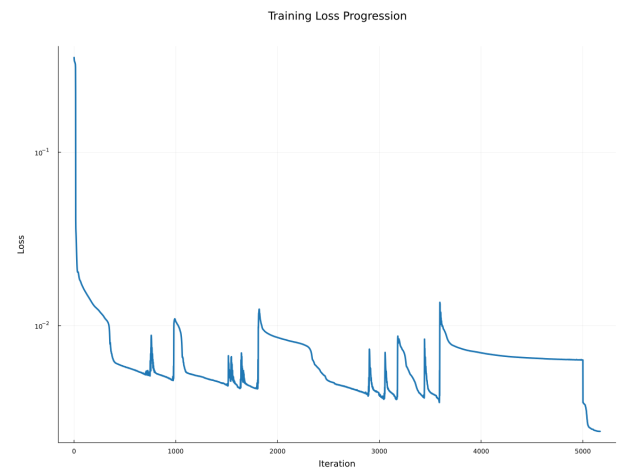


Fig. 5: Phase 2 training loss showing convergence to lower loss than Phase 1.

7.3 Phase 2: Structured-Parameter UDE

The structured-parameter UDE achieves $R^2 = 0.991$ on training data (Figure 5), with strong correlation between predicted and observed volumes (Figure 6). Figure 7 shows a representative case (C4/T2) where the model captures both immune-mediated suppression and growth dynamics; the separation between immune-present and immune-absent trajectories demonstrates biological consistency. Component decomposition (Figure 8) reveals Gompertz growth dominating early phases with immune killing peaking around days 15–20. For direct comparison with classical models, Table 1 reports R^2 values for the Gompertz baseline ($R^2 = 0.763$), Kuznetsov–Taylor ($R^2 = 0.821$), and de Pillis–Raduns kaya ($R^2 = 0.845$), all fitted to the same dataset. The structured-parameter UDE ($R^2 = 0.991$) substantially outperforms all classical baselines, indicating that the neural-parameterized dynamics capture variance that fixed-parameter mechanistic models cannot.

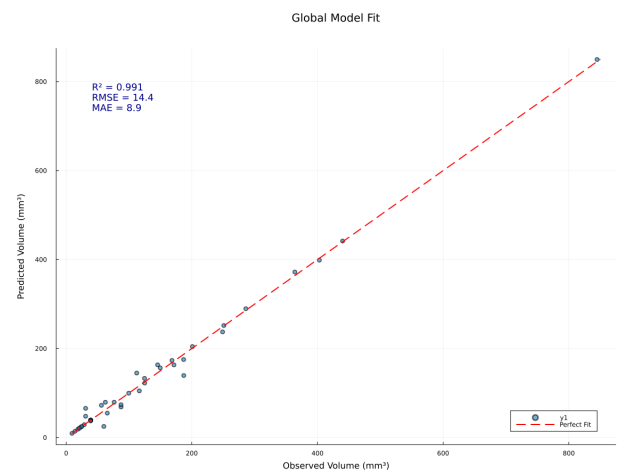


Fig. 6: Global fit for the structured-parameter UDE ($R^2 = 0.991$).

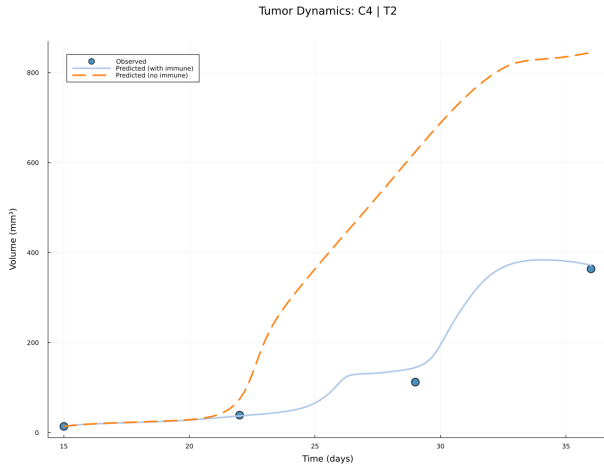


Fig. 7: Representative group (C4|T2) showing observed data, predicted trajectories, and counterfactual predictions without immune influence.

Group	MAE (mm ³)	RMSE (mm ³)	sMAPE (%)	R ²	Final Vol. Err. (%)
C2-T1	30.5	30.5	16.8	0.909	16.8
C2-T2	17.2	17.2	8.4	0.915	8.4
C2-T3	8.7	8.7	6.7	0.947	6.7
C2-T4	0.9	0.9	1.1	0.958	1.1
C4-T5	16.5	16.5	8.7	0.998	8.7
Mean	14.8	14.8	8.3	0.945	8.3
Std	11.4	11.4	5.7	0.034	5.7

Table 2.: Cross-validation metrics for the structured-parameter UDE across held-out groups. Note that C2-T2 and C4-T5 have similar RMSE values (17.2 vs. 16.5 mm³) but substantially different R^2 (0.915 vs. 0.998). This divergence arises because R^2 is normalized by the total variance of the observed data: C4-T5 exhibits much larger volume variation (higher $\text{Var}(V^{\text{obs}})$), so the same absolute error yields a higher R^2 . In contrast, C2-T2 has a narrower dynamic range, making its R^2 more sensitive to residual errors of comparable magnitude.

Figure 9 shows representative forecasting results for the C4-T5 group. Figure 10 presents both short-term (14-day) and long-term (90-day) forecast horizons with and without immune activity, illustrating the model's ability to quantify immune-mediated suppression effects across temporal scales.

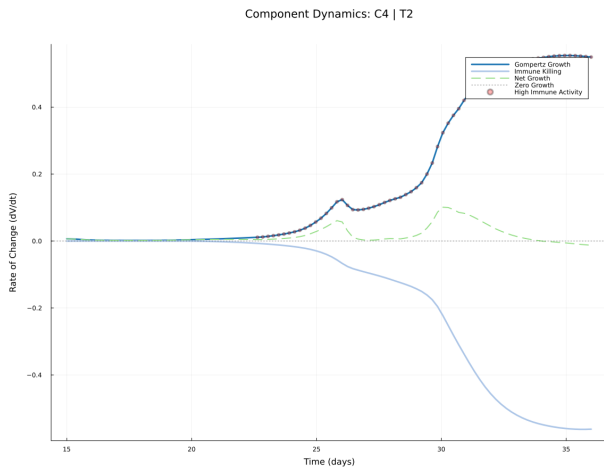


Fig. 8: Component dynamics for C4|T2: Gompertz growth, immune killing, and net growth.

7.4 Cross-Validation and Forecasting

Table 2 reports cross-validation metrics across all five held-out groups. The structured-parameter UDE achieves mean $R^2 = 0.945$ (std = 0.034) and mean sMAPE = 8.3%, indicating robust generalization.

Forecast for C4-T5

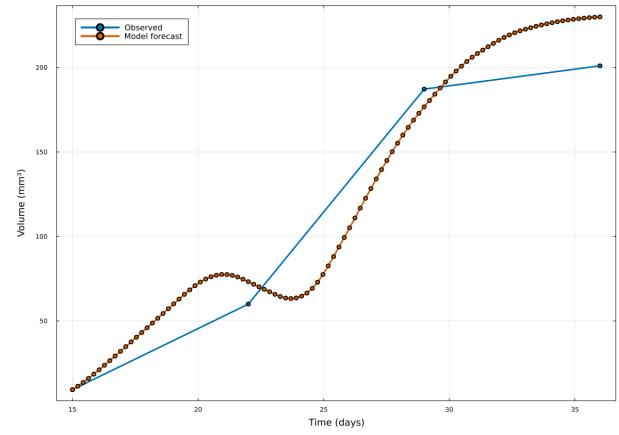


Fig. 9: Forecasting performance for C4-T5 showing agreement between observed and predicted volumes.

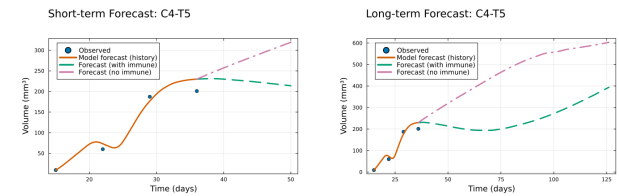


Fig. 10: Forecasting for C4-T5 with and without immune activity. Left: short-term (14-day) horizon. Right: long-term (90-day) horizon. The divergence between immune-present and immune-absent trajectories quantifies the cumulative effect of immune suppression.

The temporal evolution of learned parameters (Figure 11) shows how growth rates, carrying capacities, and immune parameters adapt over time, providing biological insight into tumor–immune interaction dynamics. We plot C4-T5 as it is the representative case used throughout the cross-validation analysis (achieving the highest held-out $R^2 = 0.998$ in Table 2).

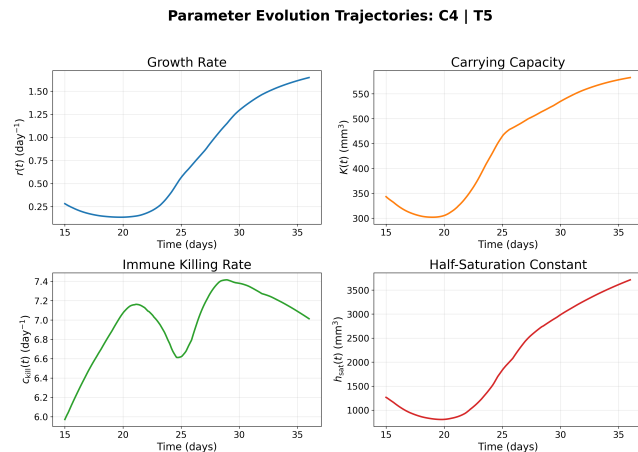


Fig. 11: Parameter evolution trajectories for the representative case C4-T5. Each subpanel shows one learned parameter as a function of time within the C4-T5 forecast window: top-left, growth rate $r(t)$ (day^{-1}); top-right, carrying capacity $K(t)$ (mm^3); bottom-left, immune killing rate $c_{\text{kill}}(t)$ (day^{-1}); bottom-right, half-saturation constant $h_{\text{sat}}(t)$ (mm^3). Each curve is the corresponding parameter for this single group; there are no series labeled “y1” (axis labels and panel titles identify the quantity directly). Fonts have been enlarged for publication readability.

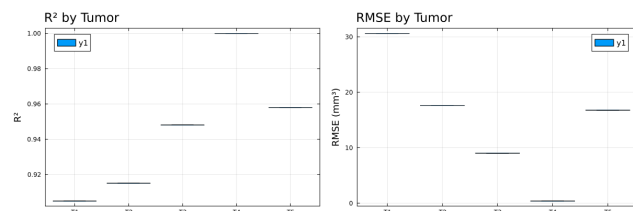


Fig. 12: Cross-validation metric distributions. Left: R^2 values across held-out folds. Right: RMSE values across held-out folds.

7.5 Computational Performance

Training takes 15–30 minutes per cross-validation fold on standard hardware (Intel i7, 16GB RAM). Inference is sub-millisecond per trajectory. Julia’s SciML ecosystem provides efficient adjoint methods for gradient computation [23, 19].

8. Discussion

The progressive workflow from hybrid to structured-parameter UDE demonstrates a practical research pattern: the hybrid model’s flexibility reveals what dynamics matter (complex immune responses, temporal corrections), while the structured-parameter model channels these insights into an interpretable framework. The structured-parameter UDE’s higher accuracy ($R^2 = 0.991$ vs. 0.897) suggests that constraining neural networks to output biologically meaningful parameters acts as effective regularization.

The cross-validation results (mean $R^2 = 0.945$) indicate reasonable generalization, though we note the dataset is small (five held-out groups from a single experimental system). The gap between training R^2 (0.991) and cross-validation R^2 (0.945) suggests some overfitting, which is expected given the model’s expressiveness relative to dataset size.

From a software perspective, Julia’s SciML ecosystem provides a cohesive development experience. The composability of DifferentialEquations.jl, Flux.jl, and SciMLSensitivity.jl enables rapid prototyping of UDE architectures, and the performance characteristics (sub-millisecond inference, efficient adjoint gradients) are well-suited to iterative model development.

8.1 Limitations

This study has several important limitations that should be considered when interpreting the results.

Limited baselines. Our comparison includes only classical mechanistic ODE models (Gompertz, Kuznetsov–Taylor, de Pillis–Radunskaya). We do not compare against modern data-driven baselines such as Neural ODEs [5], latent ODE models, or Gaussian process regression, which could provide competitive or superior performance on this dataset. Future work should include these comparisons to properly contextualize the UDE approach.

No ablation studies. We do not systematically ablate model components (e.g., removing individual loss terms, varying network size, testing different mechanistic backbones). Such studies would clarify which design choices contribute most to performance and are an important direction for future work.

No uncertainty quantification. Our models produce point predictions without confidence intervals. For any downstream application, uncertainty estimates are essential. Techniques such as ensemble methods, Monte Carlo dropout, or Bayesian neural network extensions could provide this, but are not implemented here.

Extrapolation beyond training range. Because the Phase 2 parameter networks take normalized time t as a direct input, learned parameter functions may not generalize well beyond the training temporal window. The sigmoid-bounded activations mitigate divergence, but long-term forecasts should be treated as extrapolations with increasing uncertainty.

Naming and interpretability. While we use the term “structured-parameter” to describe the Phase 2 architecture, it is important to note that neural networks learn all four biological parameters as functions of state. The model is not mechanistic in the traditional sense—it is a neural model whose outputs are structured through a mechanistic template. The biological interpretability of individual parameter trajectories should be treated with caution.

Dataset scope. The dataset comes from a single experimental system with a small number of tumor–immune trajectories. Generalization to other tumor types, species, or experimental conditions has not been demonstrated. The single-compartment model also neglects spatial heterogeneity and metastatic dynamics.

Forecasting caveats. The reported forecasting metrics are computed on this specific dataset and should not be interpreted as indicative of performance on independent datasets or in applied settings. Long-term forecasts beyond the training data’s temporal extent are extrapolations and should be treated accordingly.

9. Conclusions

We have presented a progressive UDE workflow for tumor–immune dynamics modeling in Julia’s SciML ecosystem. The dual-network hybrid UDE ($R^2 = 0.897$) serves as an exploratory tool, while

the structured-parameter UDE ($R^2 = 0.991$, cross-validation mean $R^2 = 0.945$) demonstrates that channeling neural network outputs through classical equation structures can yield both high accuracy and parameter-level interpretability. The implementation showcases the capabilities of Julia's SciML ecosystem for composing differential equations with neural networks in a scientifically grounded manner. Important next steps include comparison against modern neural baselines, ablation studies, uncertainty quantification, and validation on larger and more diverse datasets.

10. Research impact statement

This work demonstrates the application of Universal Differential Equations to tumor-immune dynamics modeling within Julia's SciML ecosystem. The progressive workflow from hybrid to structured-parameter models provides a replicable template for applying UDEs to biological systems. The physics-informed constraints and cross-validation framework contribute methodological tools that are transferable to other domains. The open-source implementation supports reproducible research in computational biology.

11. AI usage disclosure

AI-assisted tools (GitHub Copilot and Claude by Anthropic) were used during development for code assistance, debugging, and manuscript editing. All content was reviewed, verified, and revised by the author. The scientific methodology, experimental design, model architectures, and interpretation of results are the work of the author.

12. References

- [1] Doaa Aboelyazeed, Chonggang Xu, Forrest M. Hoffman, Jiangtao Liu, Alex W. Jones, Chris Rackauckas, Kathryn Lawson, and Chaopeng Shen. A differentiable, physics-informed ecosystem modeling and learning framework for large-scale inverse problems: Demonstration with photosynthesis simulations. *Biogeosciences*, 20(13):2671–2692, 2023. doi:10.5194/bg-20-2671-2023.
- [2] Kevin Atsou, Fabienne Anjuere, Veronique M. Braud, and Thierry Goudon. A size and space structured model of tumor growth describes a key role for protumor immune cells in breaking equilibrium states in tumorigenesis. *PLOS ONE*, 16(11):e0259291, 2021. doi:10.1371/journal.pone.0259291.
- [3] Kevin Atsou, Sokchea Khou, Fabienne Anjuere, Veronique M. Braud, and Thierry Goudon. Analysis of the equilibrium phase in immune-controlled tumors provides hints for designing better strategies for cancer treatment. *Frontiers in Oncology*, 12:878827, 2022. doi:10.3389/fonc.2022.878827.
- [4] Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B. Shah. Julia: A fresh approach to numerical computing. *SIAM Review*, 59(1):65–98, 2017. doi:10.1137/141000671.
- [5] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. doi:10.48550/arXiv.1806.07366.
- [6] Raj Dandekar, Chris Rackauckas, and George Barbastathis. A machine learning-aided global diagnostic and comparative tool to assess effect of quarantine control in COVID-19 spread. *Patterns*, 1(9):100145, 2020. doi:10.1016/j.patter.2020.100145.
- [7] Lisette G. de Pillis, Ami E. Radunskaya, and Charles L. Wiseman. A validated mathematical model of cell-mediated immune response to tumor growth. *Cancer Research*, 65(17):7950–7958, 2005. doi:10.1158/0008-5472.CAN-05-0564.
- [8] Chris Finlay, Jörn-Henrik Jacobsen, Levon Nurbekyan, and Adam M. Oberman. How to train your neural ODE. 2020. doi:10.48550/arXiv.2002.02798. 2002.02798.
- [9] Mike Innes. Flux: Elegant machine learning with Julia. *Journal of Open Source Software*, 3(25):602, 2018. doi:10.21105/joss.00602.
- [10] Mike Innes, Alan Edelman, Keno Fischer, Chris Rackauckas, Elliot Saba, Viral B. Shah, and Will Tebbutt. A differentiable programming system to bridge machine learning and scientific computing. 2019. doi:10.48550/arXiv.1907.07587. 1907.07587.
- [11] George Em Karniadakis, Ioannis G. Kevrekidis, Lu Lu, Paris Perdikaris, Sifan Wang, and Liu Yang. Physics-informed machine learning. *Nature Reviews Physics*, 3(6):422–440, 2021. doi:10.1038/s42254-021-00314-5.
- [12] Vladimir A. Kuznetsov, I. A. Makalkin, M. A. Taylor, and Alan S. Perelson. Nonlinear dynamics of immunogenic tumors: parameter estimation and global bifurcation analysis. *Bulletin of Mathematical Biology*, 56(2):295–321, 1994. doi:10.1007/BF02460644.
- [13] Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3):218–229, 2021. doi:10.1038/s42256-021-00302-5.
- [14] Stefano Massaroli, Michael Poli, Jinkyoo Park, Atsushi Yamashita, and Hajime Asama. Dissecting neural ODEs. *Advances in Neural Information Processing Systems*, 33:3952–3963, 2020. doi:10.48550/arXiv.2002.08071.
- [15] Chris Rackauckas, Mike Innes, Yingbo Ma, et al. DiffEqFlux.jl – a julia library for neural differential equations, 2019. doi:10.48550/arXiv.1902.02376. 1902.02376.
- [16] Chris Rackauckas and Qing Nie. DifferentialEquations.jl – a performant and feature-rich ecosystem for solving differential equations in Julia. *Journal of Open Research Software*, 5(1):15, 2017. doi:10.5334/jors.151.
- [17] Christopher Rackauckas, Yingbo Ma, Julius Martensen, Collin Warner, Kirill Zubov, Rohit Supekar, Dominic Skinner, Ali Ramadhan, and Alan Edelman. Universal differential equations for scientific machine learning. 2020. doi:10.48550/arXiv.2001.04385. 2001.04385.
- [18] Maziar Raissi, Paris Perdikaris, and George E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019. doi:10.1016/j.jcp.2018.10.045.
- [19] Ali Ramadhan, John Marshall, Andre Souza, Xin Kai Lee, Ulyana Piterbarg, Adeline Hillier, Gregory LeClaire Wagner, Christopher Rackauckas, Chris Hill, Jean-Michel Campin, and Raffaele Ferrari. Capturing missing physics in climate model parameterizations using neural differential equations. 2020. doi:10.48550/arXiv.2010.12559. 2010.12559.
- [20] Chaopeng Shen, Alison P. Appling, Pierre Gentine, Toshiyuki Bandai, Hoshin Gupta, Alexandre Tartakovsky, Marco Baity-Jesi, Fabrizio Fenicia, Daniel Kifer, Li Li, et al. Differentiable modelling to unify machine learning and physical models for geosciences. *Nature Reviews Earth & Environment*, 4(8):552–567, 2023. doi:10.1038/s43017-023-00450-9.

- [21] Takeshi Teshima, Koichi Tojo, Masahiro Ikeda, Isao Ishikawa, and Kenta Oono. Universal approximation property of neural ordinary differential equations. 2020. doi:10.48550/arXiv.2012.02414. 2012.02414.
- [22] Cristina Vaghi, Alix Rodallec, Raphaelle Fanciullino, Joseph Ciccolini, Jonathan P. Mochel, Michalis Matri, Clair Poignard, John M. L. Ebos, and Sebastien Benzekry. Population modeling of tumor growth curves and the reduced gompertz model improve prediction of the age of experimental tumors. *PLOS Computational Biology*, 16(2):e1007178, 2020. doi:10.1371/journal.pcbi.1007178.
- [23] Allen M. Wang, Darren T. Garnier, and Cristina Rea. Hybridizing physics and neural ODEs for predicting plasma inductance dynamics in tokamak fusion reactors. 2023. doi:10.48550/arXiv.2310.20079. 2310.20079.